

SEPTEMBER 25, 2025

GUIDED PROJECT

SAAS-BASED CAR BOOKING SYSTEM (SAAS ON ZOHO CREATOR)

DR C RAJESH BABU

PROFESSOR

CAR BOOKING SYSTEM (SAAS ON ZOHO CREATOR)

Tools: Browser + Zoho Creator free account

Concepts: SaaS, cloud DB design, forms, lookups, form rules, reports, dashboards, roles/portal

Tools

- **Browser:** Google Chrome or Edge
- **Zoho Creator Free Account:** <https://creator.zoho.com>

Core Concepts

SaaS • Cloud Database Design • Forms • Lookups • Form Rules • Reports • Dashboards • Roles & Portal (Multi-Tenancy)

Aim

To build a **cloud-based Car Booking Application** where:

- **Admins** add branches, cars, and view analytics
- **Customers** browse and book cars online
- **Staff** manage bookings and payments

All accessed securely via browser, with role-based permissions – *no code required*.

What You'll Build

Component	Purpose
Forms	Branches, Cars, Customers, Bookings, Payments
Logic	Field properties, form rules, conditional visibility, default values, lookup filters, formula fields
Reports	Available Cars, All Bookings, My Bookings, Closed Bookings
Dashboard	KPIs + Charts + Embedded Reports
SaaS Setup	Roles, Permissions, and Customer Portal (multi-tenant access)

STEP 0 – Create the App

Instructions

1. Go to <https://creator.zoho.com>
2. Click **Create New Application** → **Create from Scratch**
3. Enter app name:
Car Booking SaaS
4. Click **Create**

Student Note:

This creates your *own cloud workspace* – your app lives on Zoho's servers.
Everyone logs in via browser → This is **SaaS (Software as a Service)** in action.

STEP 1 – Create Forms (Your Cloud Database)

Each **Form** = one **Table** in a database.

Each **Field** = a **Column**.

Each **Record** = a **Row**.

1.1 Branches

Purpose: Store details of car rental branches.

Steps

1. Left Panel → **Create** → **Form** → **Blank Form**
2. Name the form **Branches**
3. Add fields in this order:

Field Name	Type	Required	Example
BranchID	Auto Number	✓	Auto-generated
BranchName	Single Line	✓	"Guindy"
City	Single Line		"Chennai"
Phone	Number		9876543210

Save

Learning Note:

This acts as a *master table* for all your branches – cars and bookings will reference this later via **lookups**.

1.2 Cars

Purpose: Store car details linked to a branch.

Steps

○ Create → Form → Blank Form → Name it **Cars**

1. Add fields:

Field	Type	Required	Example
CarID	Auto Number	✓	Auto
Registration No	Single Line	✓ (unique)	TN10AB1234
Model	Single Line	✓	"Swift"
CarType	Dropdown	✓	Hatchback, Sedan, SUV, Luxury, Van
Transmission	Dropdown		Manual / Automatic
DailyRate	Currency	✓	₹2500
Branch	Lookup → Branches → BranchName	✓	Guindy
Available	Checkbox		✓ = Available
Images	File Upload (multiple)		Upload car photos
Notes	Multi Line		"Diesel, AC"

 **Save**

Concept Note:

A **Lookup Field** links data between forms – like a *foreign key* in DBMS. So, every car now "belongs" to a Branch.

1.3 Customers

Purpose: Store customer information.

Steps

1. Create → Form → Blank Form → Name: **Customers**
2. Add fields:

Field	Type	Required
CustomerID	Auto Number	✓
FullName	Single Line	✓
Email	Email	✓
Phone	Number	
IDProof	File Upload	

 **Save**

Learning Note:

This form will later link to **Bookings** through a lookup.
In a real SaaS portal, this data ties to the **logged-in user** identity.



1.4 Bookings

Purpose: Store booking details – who booked which car, when, and where.

Steps

1. Create → Form → Blank Form → Name: **Bookings**
2. Add fields:

Field	Type	Required	Example
BookingID	Auto Number	✓	Auto
Customer	Lookup → Customers → FullName	✓	Arjun
Car	Lookup → Cars → RegistrationNo	✓	TN10AB1234

PickupBranch	Lookup → Branches → BranchName	✓	Guindy
DropoffBranch	Lookup → Branches → BranchName	✓	Tambaram
PickupDateTime	Date-Time	✓	15/10/2025 09:00
DropoffDateTime	Date-Time	✓	17/10/2025 09:00
Days	Formula (Number)	✓	Auto-calculated
DailyRate	Currency	✓	Auto-fetched
TotalCost	Formula (Currency)	✓	Auto-calculated
Status	Dropdown	✓	Pending, Confirmed, Cancelled, Completed
Notes	Multi Line		

Now Add No-Code Logic (All GUI Based)

a. Car Lookup Filter

In Car field → Properties → "Filter Records" →

Condition: Show only records where Available = true.

 **Concept:** Lookup Filters = Data Validation without code.

b. Auto-fill Daily Rate

In DailyRate → Default Value → Choose **Car.DailyRate**.

 Snapshotting rate keeps consistent price even if car price later changes.

c. Days Formula

Formula (Number):

DateDiff(DropoffDateTime, PickupDateTime, "days", true)

d. TotalCost Formula

Formula (Currency):

Days * DailyRate



Save Form

**Student Note:**

You've now connected **four entities** (Customers-Cars-Branches-Bookings) using lookups and formulas – like a relational model, but no code!

**1.5 Payments (Optional)**

Purpose: Record payments for bookings.

**Steps**

Form → Blank Form → Name: **Payments**

Add:

Field	Type
PaymentID	Auto Number
Booking	Lookup → Bookings → BookingID
Amount	Currency
Mode	Dropdown (UPI, Card, Cash)
PaidOn	Date-Time (default = Now)
Paid	Checkbox



Save



Note: Payments are optional but helpful for revenue tracking.

STEP 2 – Form Rules (No Code Validations)

Objective

Prevent wrong data and improve user experience.

Where: Form → **Workflows** → **Form Rules** → **Add Rule**

Rule 1 – Drop-off after Pickup

Form: **Bookings**

Trigger: On User Input

Condition: DropoffDateTime ≤ PickupDateTime

Action: Show message “Drop-off must be after pickup!” and **Disallow submission.**

 *Concept:* Client-side validation stops bad data before saving to the cloud DB.

Rule 2 – Mandatory Branches

Form: **Bookings**

Trigger: On Validate

Action: Make PickupBranch and DropoffBranch mandatory.

Rule 3 – Default Booking Status

Set **default value** of Status field = *Pending* (via field properties).

 *Concept:*

Rules = automation without coding.

These make your app **intelligent at the form level.**

STEP 3 – Role-Based Access (SaaS Multi-Tenancy)

Objective

Give different access to Admins, Staff, and Customers – all inside one app.

Steps

1. Go to **Settings** → **Users & Permissions** → **Roles**

2. Add Roles:

- ☐ **Admin**
- ☐ **Staff**
- ☐ **Customer**

Role	Permissions
Admin	Full access to all forms and reports
Staff	Read/Write Cars, Bookings, Payments; Read Customers
Customer	View Available Cars, Create & Read their own Bookings only

Enable Customer Portal

Settings → **Portal** → **Enable**

Then:

- Invite customer emails under **Customer Role**
- In **My Bookings** report (next step), filter records where:
Customer = Logged-in User

 Concept Note:

One shared app, isolated data per user = **Multi-Tenancy**, core to SaaS.



STEP 4 – Reports (Visual Data Views)

 Objective

Turn forms into filtered, sortable, shareable views.



4.1 Available Cars

- Source: Cars
- Report Type: **List Report** → Name: *Available Cars*
- Filter: Available = true
- Columns: RegistrationNo, Model, CarType, Transmission, DailyRate, Branch
- Enable Quick Filters: CarType, Branch
- Enable Search: Model, RegistrationNo

 *Concept:* Report = Saved Query in cloud DB.



4.2 All Bookings

- Source: Bookings → List Report → Name: *All Bookings*
- Columns: BookingID, Customer, Car, PickupDateTime, DropoffDateTime, Days, TotalCost, Status
- Group by: Status
- Quick Filters: Status, PickupBranch

4.3 My Bookings (Customer View)

- Duplicate All Bookings → Rename: *My Bookings*
- Filter: Customer = Logged-in User
- Hide unnecessary fields (TotalCost optional).

 *Concept:* Personalised view = Portal-based SaaS behavior.

4.4 Closed Bookings

- Duplicate All Bookings → Rename: *Closed Bookings*
- Filter: Status IN (Cancelled, Completed)

4.5 Payments (Optional)

- Source: Payments → List Report → Name: *Payments Received*
- Enable **Aggregate** → **SUM(Amount)** to show total revenue.

STEP 5 – Dashboards (Analytics for SaaS)

Objective

Create a visual command center for operations.

Steps

1. Create → Page → Dashboard → Name: **Operations Dashboard**
2. Add **KPI Widgets**:
 - ☐ Total Cars Available → Count of Cars where Available = true
 - ☐ Active Bookings → Count of Bookings where Status in (Pending, Confirmed)
 - ☐ (Optional) Total Revenue (This Month) → from Payments report
3. Add **Charts**:
 - ☐ Bar Chart → Bookings by CarType
 - ☐ Pie Chart → Booking Status Distribution
 - ☐ Line Chart → Bookings by Pickup Date
4. Add **Embedded Reports**:
 - ☐ Available Cars
 - ☐ All Bookings or My Bookings

**Concept Note:**

Dashboards = live analytics built from your cloud data – *no spreadsheet needed.*



STEP 6 – Sample Data & Dry Run

**Data Entry**

1. Add **2-3 Branches** (Guindy, Tambaram, T Nagar)
2. Add **8-10 Cars** (mixed CarTypes, Available = true)
3. Add **3-4 Customers**
4. Log in as Customer → Open *Available Cars* → Create 2 Bookings
5. Log in as Staff → Open *All Bookings* → Change Status = Confirmed

- (Optional) Add Payments for each booking

✓ Verify

- Dashboard KPIs show correct numbers
- *My Bookings* shows only logged-in user's records
- Available Cars updates when a car is booked (Available = false)

🧠 Concept:

Testing shows real-world SaaS workflows using live cloud data.



STEP 7 – Share & Deploy

- Go to **Share → Application**
- Invite users under their roles (Admin / Staff / Customer)
- Open app on a mobile browser to test responsiveness

✓ *Concept:* Deployment = instantly live for all users, just like any SaaS.



Final Output – What Should Work

Role	What They Can Do
Customer	View cars, make bookings, see only their bookings
Staff	Manage cars, confirm bookings, mark payments
Admin	Access dashboards and all data

🧠 No code written – all logic through GUI settings.

You've built a **SaaS Car Booking Platform** using **cloud database principles**.



Concept Notes Recap

Term	Meaning in This Project
Form	Table in cloud DB
Field	Column
Record	Row
Lookup	Relationship (Foreign Key)
Form Rule	No-code automation
Formula Field	Computed value (like a virtual column)
Report	Saved, filterable view of data
Dashboard	Real-time analytics from reports
Role/Portal	Multi-tenant access (different views per user)



Learning Outcomes

By completing this project, students will:

- ✓ Design a **relational cloud database**
- ✓ Apply **data validation rules** using GUI tools
- ✓ Create **linked forms & lookup relationships**
- ✓ Build **reports and dashboards** for analytics
- ✓ Configure **multi-role SaaS access** without code
- ✓ Explain how **no-code tools** implement DBMS + SaaS concepts together

Challenge Tasks (Pick Any 5)

Task	Concept Reinforced
1. “Available Cars by Branch” report	Lookup filters + analytics
2. Disallow >30-day booking (Form Rule)	Validation logic
3. Add “WeekendRate” field	Dynamic pricing awareness
4. Kanban View on Cars (Available/Unavailable)	Visual data grouping
5. Customer “My Profile” report	Portal usability
6. Cancellations Pie Chart	Analytical reporting
7. Revenue KPI (This Month)	Aggregates & filters
8. Pickup Calendar View	Visualization by date
9. Photo Gallery of Cars	UI customization
10. “I Agree to Terms” checkbox	Mandatory acceptance via rules



Suggested Submission Pack

- ☒ App URL + Demo Logins (Admin / Staff / Customer)
- ☒ 6-10 Screenshots (Forms, Reports, Dashboard)
- ☒ One-page Write-up:
“How our app demonstrates SaaS principles (multi-tenant access, cloud DB, reports, dashboards, zero install)”